Institut National des Postes et Télécommunications

Rabat, Maroc

Projet Python

Etude comparative des algorithmes de data augmentation CTGAN, TGAN et TVAE , suivant leurs scores sur des modèles de Machine Leaning et Deep Learning

Génération de données synthétiques tabulaires

pour la détection comportementale de logiciels malveillants

Groupe

Awi Detine Jean Arthur

Guindo Ibrahim

Koujane Ismaïl

Encadrant

. Kamal Idrissi Hamza

Année universitaire

2025–2026

Résumé

Méthodologie de travail

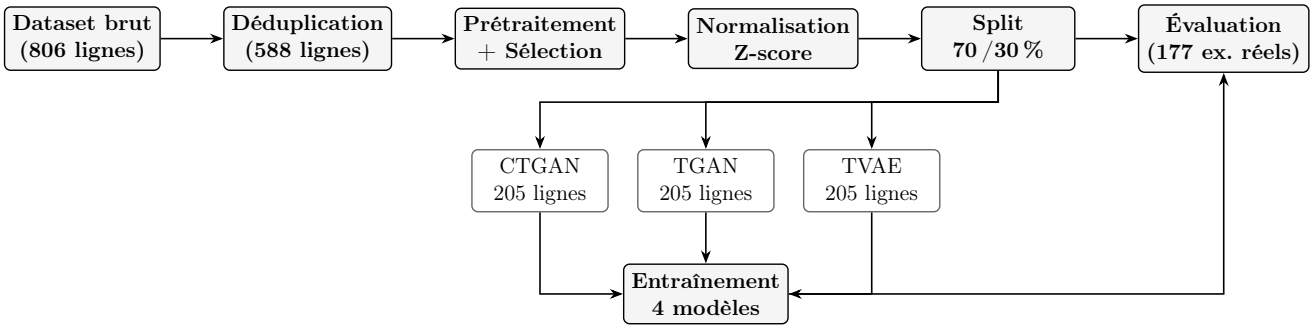

Figure 1 – Pipeline global – méthodologie de travail

La détection des ransomwares est l'un des défis majeurs de la cybersécurité moderne. Ces logiciels malveillants, qui chiffrent les fichiers de leurs victimes et exigent une rançon, ont causé des pertes considérables dans de nombreuses organisations. En 2025, les dégâts mondiaux liés aux ransomwares sont estimés à plus de 30 milliards de dollars, avec une attaque toutes les 11 secondes. Dans ce contexte, l'analyse comportementale basée sur les appels d'API Windows constitue une approche prometteuse pour détecter ces menaces, même dans leurs variantes inconnues.

Ce projet explore l'apport de l'augmentation de données synthétiques pour améliorer les performances des modèles de classification de logiciels malveillants. À partir d'un dataset de **806 exécutables** (588 après déduplication), caractérisés par leurs appels d'API Windows, nous appliquons trois algorithmes de génération : **CTGAN** (Conditional Tabular GAN), **TGAN** (via `GaussianCopulaSynthesizer`, alternative au modèle original déprécié) et **TVAE** (Tabular Variational AutoEncoder).

Après un prétraitement rigoureux comprenant la déduplication, la suppression des variables à forte proportion de valeurs nulles (seuil 90 %), un test de Mann-Whitney pour éliminer les features non discriminantes, une analyse de corrélation de Spearman avec suppression des redondances (seuil 0,9), une sélection supervisée par `SelectKBest` associée à une recherche par validation croisée, et enfin une **normalisation Z-score**, chaque générateur produit 205 lignes synthétiques qui sont ajoutées au jeu d'entraînement de 411 lignes originales. Quatre modèles de classification sont évalués sur un jeu de test fixe de **177 exemples réels** : Kernel SVM, KNN, MLP (réseau dense 256-128-64) et CNN 1D (deux couches convolutionnelles).

Les résultats obtenus montrent que **TVAE produit les données synthétiques les plus fidèles** (77,8 % des distributions conservées au test de Kolmogorov-Smirnov). La combinaison **MLP + TVAE atteint l'AUC maximale** (0,9223). Le **Kernel SVM maximise le recall** (0,9878), métrique prioritaire en cybersécurité car un faux négatif (ransomware non détecté) est bien plus coûteux qu'un faux positif. La meilleure combinaison F1/précision est réalisée par **MLP + TGAN** (F1 = 0,8404) et **KNN + TVAE** (précision = 0,8462, accuracy = 0,8418). Le CNN 1D, bien que sensible à la qualité des données, affiche des performances honorables sur le jeu original (AUC 0,8786) et bénéficie partiellement de la haute fidélité de TVAE (AUC 0,7964).

L'étude met en évidence l'importance cruciale de la normalisation Z-score pour les modèles sensibles à l'échelle (SVM, CNN) et pour la qualité des données synthétiques. Elle confirme que l'augmentation de données par modèles génératifs est une stratégie fiable pour enrichir des jeux de données restreints en cybersécurité, à condition de choisir le générateur adapté à l'objectif (recall maximal, AUC, F1, etc.).

Mots-clés : ransomware, appels d'API Windows, augmentation de données, CTGAN, TGAN, TVAE, normalisation Z-score, classification, Kernel SVM, KNN, MLP, CNN, cybersécurité.

Abstract — Ransomware detection is a critical challenge in modern cybersecurity. This project explores synthetic data augmentation to improve malware classification based on Windows API calls. Starting from 806 executables (588 after deduplication), we apply CTGAN, TGAN and TVAE after a rigorous preprocessing pipeline including Z-score normalization. Four classifiers (Kernel SVM, KNN, MLP, 1D-CNN) are evaluated on a fixed 177-sample real test set. TVAE achieves the best statistical fidelity (77.8% KS). MLP+TVAE reaches the best AUC (0.9223), MLP+TGAN the best F1 (0.8404), and Kernel SVM the best recall (0.9878).

Table des matières

Résumé **1**

1 Introduction **6**

1.1 Contexte et enjeux de la cybersécurité 6

1.2 Limites des approches traditionnelles 6

1.3 Détection comportementale par appels d'API Windows 7

1.4 Objectifs du projet 7

1.5 Organisation du rapport 8

2 Principe et point rappel sur les modèles utilisés **9**

2.1 Détection comportementale des ransomwares 9

2.1.1 Principes généraux 9

2.1.2 Appels d'API Windows comme features 9

2.2 Méthodes d'augmentation de données tabulaires 9

2.2.1 Problématique de la petite taille des datasets 10

2.2.2 CTGAN (Conditional Tabular GAN) 10

2.2.3 TGAN et son alternative moderne 10

2.2.4 TVAE (Tabular Variational AutoEncoder) 11

2.2.5 Comparaison théorique 11

3 Présentation et analyse du Dataset **12**

3.1 Description générale 12

3.1.1 Statistiques initiales 12

3.2 Analyse exploratoire (EDA) 12

3.2.1 Détection et suppression des doublons 12

3.2.2 Analyse des variables quasi-nulles 13

3.2.3 Histogrammes comparatifs par classe 13

3.2.4 Test de Mann-Whitney pour la significativité 14
3.2.5 Analyse de corrélation de Spearman 14
3.3 Sélection supervisée des features 15
3.4 Évolution du dataset 16

4 Méthodologie **17**
4.1 Pipeline global 17
4.2 Normalisation Z-score 17
4.3 Protocole d'évaluation 18
4.4 Métriques d'évaluation 18

5 Modèles de classification **19**
5.1 Kernel SVM (Support Vector Machine) 19
5.2 K-Nearest Neighbors (KNN) 19
5.3 MLP (Multi-Layer Perceptron) 19
5.4 CNN 1D (Convolutional Neural Network 1D) 20

6 Résultats expérimentaux **21**
6.1 Qualité des données synthétiques 21
6.1.1 Équilibre des classes 21
6.1.2 Fidélité statistique : test KS 22
6.2 Performances des modèles sur le jeu de test réel 22
6.2.1 Baseline : données originales (411 échantillons) 22
6.2.2 Augmentation avec CTGAN 23
6.2.3 Augmentation avec TGAN 23
6.2.4 Augmentation avec TVAE 24
6.3 Comparaison globale 24
6.4 Analyse et discussion 26
6.4.1 Impact de la normalisation Z-score 26
6.4.2 Choix du générateur selon l'objectif 26
6.4.3 Sensibilité du CNN 26
6.4.4 Limites de l'étude & perspectives 27

7 Conclusion et perspectives **28**

7.1 Conclusion 28

A Partie du Code source demandée **29**

Chapitre 1

Introduction

1.1 Contexte et enjeux de la cybersécurité

La transformation numérique des entreprises et des administrations s'accompagne d'une exposition accrue aux cybermenaces. Parmi celles-ci, les **ransomwares** occupent une place prépondérante en raison de leur impact direct sur la disponibilité des données et des systèmes. Un ransomware est un logiciel malveillant qui, une fois exécuté, chiffre les fichiers de la victime et demande le paiement d'une rançon, généralement en cryptomonnaie, pour fournir la clé de déchiffrement.

Les attaques par ransomware ont connu une recrudescence ces dernières années, touchant des hôpitaux (Centre Hospitalier de Corbeil-Essonnes en 2025), des collectivités territoriales (Commune de Villeurbanne), des entreprises (Colonial Pipeline aux États-Unis) et même des infrastructures critiques. Le coût médian d'une attaque pour une entreprise est désormais supérieur à 1,5 million de dollars, incluant la rançon elle-même, les pertes d'exploitation, les frais de remise en état et les pénalités réglementaires.

1.2 Limites des approches traditionnelles

Les antivirus traditionnels reposent essentiellement sur la **détection par signatures** : ils comparent les fichiers analysés à une base de signatures connues de malwares. Cette méthode est inefficace face aux **variantes polymorphes** (qui changent leur code à chaque infection) et aux **zero-day** (malwares inconnus des bases de signatures). De plus, les ransomwares modernes utilisent des techniques d'évasion (injection de code, exécution en mémoire seulement) qui leur permettent de contourner ces outils.

1.3 Détection comportementale par appels d'API Windows

Face à ces limites, la **détection comportementale** émerge comme une alternative robuste. Elle consiste à analyser le comportement d'un programme pendant son exécution, indépendamment de son code statique. Sous Windows, la grande majorité des actions d'un programme (lecture/écriture de fichiers, création de processus, accès au registre, communications réseau) se font via des appels aux API système. En enregistrant les fréquences d'appel à différentes fonctions API, on construit une empreinte comportementale numérique.

Un ransomware typique présente un profil caractéristique : appels massifs à `NtCreateFile`, `NtWriteFile`, `SetEndOfFile` (écriture de fichiers chiffrés), à `SetFileTime` (manipulation des métadonnées), et à `NtSetInformationFile` avec la classe `FileDispositionInfo` (suppression). Les goodwares, en revanche, tendent à appeler davantage des API d'information comme `GetFileVersionInfoExW` ou `NtQueryFullAttributesFile`. Cette différence permet de construire des classifieurs efficaces.

1.4 Objectifs du projet

Ce projet vise à répondre à cette problématique en explorant l'apport de **l'augmentation de données synthétiques** générées par des modèles génératifs. Plus précisément, les objectifs sont :

1. Mettre en place une chaîne de prétraitement rigoureuse (déduplication, filtrage statistique, sélection de features, normalisation) sur un dataset réel d'appels d'API.
2. Générer trois jeux de données synthétiques (CTGAN, TGAN, TVAE) de taille équivalente à 50 % du jeu d'entraînement (TrainSet), pour assurer un rapport de proportionnalité entre la taille du testset et celle du datset généré pas trop loin des standards
3. Évaluer la qualité statistique des données générées via le test de Kolmogorov-Smirnov et des visualisations t-SNE.
4. Comparer les performances de quatre modèles de classification (Kernel SVM, KNN, MLP, CNN 1D) entraînés sur les données originales et sur chaque jeu augmenté, en mesurant accuracy, recall, précision, F1 et AUC.
5. Identifier les meilleures combinaisons (modèle + générateur) selon différents critères opérationnels (recall maximal, F1 optimal, AUC maximale).

1.5 Organisation du rapport

Le chapitre 2 présente un point culture sur la détection comportementale et les méthodes de génération de données tabulaires. Le chapitre 3 décrit le dataset, son analyse exploratoire et les étapes de prétraitement. Le chapitre 4 expose la méthodologie expérimentale, y compris le protocole d'évaluation et les métriques. Le chapitre 5 détaille l'architecture des quatre modèles de classification. Le chapitre 6 présente et analyse les résultats obtenus. Enfin, le chapitre 7 conclut et propose des perspectives.

Chapitre 2

Principe et point rappel sur les modèles utilisés

2.1 Détection comportementale des ransomwares

2.1.1 Principes généraux

La détection comportementale repose sur l'hypothèse que les actions d'un logiciel malveillant diffèrent de celles d'un logiciel bénin, et que ces différences peuvent être apprises par un modèle. Contrairement à l'analyse statique (qui examine le code sans exécution), l'analyse dynamique capture l'exécution réelle dans un environnement isolé (sandbox). Les traces d'exécution peuvent prendre différentes formes : séquences d'appels système, flux réseau, accès au registre, modifications du système de fichiers, etc.

2.1.2 Appels d'API Windows comme features

Les API Windows sont des fonctions fournies par le système d'exploitation pour permettre aux programmes d'interagir avec le noyau, la mémoire, les périphériques et les fichiers. Chaque appel d'API est un événement discret. Dans la pratique, on compte le nombre d'occurrences de chaque type d'appel par exécutable, ce qui forme un vecteur de features de grande dimension (plusieurs centaines d'API). Des études antérieures (Ki et al., 2015 ; Souri et al., 2018) ont montré que les fréquences d'appel à des API spécifiques (notamment `NtCreateFile`, `NtWriteFile`, `RegSetValue`, `InternetOpenUrl`) sont fortement discriminantes.

2.2 Méthodes d'augmentation de données tabulaires

2.2.1 Problématique de la petite taille des datasets

Lorsqu'un dataset d'entraînement est petit (quelques centaines de lignes), les modèles complexes comme les réseaux de neurones profonds souffrent de surapprentissage : ils mémorisent les exemples d'entraînement sans généraliser. L'augmentation de données classique (bruit, rotation, etc.) n'est pas directement applicable aux données tabulaires. Les méthodes de sur-échantillonnage comme SMOTE (Chawla et al., 2002) génèrent des points intermédiaires entre voisins, mais elles ne capturent pas les dépendances complexes entre variables.

2.2.2 CTGAN (Conditional Tabular GAN)

Proposé par Xu et al. en 2019, CTGAN est une adaptation des GANs aux données tabulaires. Deux innovations clés :

— **Normalisation spécifique au mode** : chaque colonne numérique est modélisée comme un mélange de gaussiennes (VGM) pour gérer les distributions multimodales et éparses.

— **Vecteur conditionnel** : le générateur reçoit une condition (le label de classe) pour contrôler la distribution des classes générées.

L'objectif d'un GAN standard est donné par l'équation 2.1 :

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))] \tag{2.1}$$

CTGAN ajoute un terme de conditionnement qui permet de générer des échantillons d'une classe spécifique.

2.2.3 TGAN et son alternative moderne

TGAN (Xu & Veeramachaneni, 2018) est le prédécesseur non conditionnel de CTGAN. Il utilise une architecture GAN standard sans contrôle de la classe générée. La bibliothèque originale TGAN est dépréciée et incompatible avec Python 3.10+. Pour cette étude, nous utilisons `GaussianCopulaSynthesizer` de la bibliothèque SDV (Synthetic Data Vault), qui modélise les dépendances par une copule gaussienne et génère sans conditionnement. Cette approche est simple, rapide et produit des résultats reproductibles.

2.2.4 TVAE (Tabular Variational AutoEncoder)

Toujours issu des travaux de Xu et al. (2019), TVAE remplace l'approche adversariale par un autoencodeur variationnel. L'objectif est de maximiser la borne inférieure de la vraisemblance (ELBO) :

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{\text{KL}}(q(z|x)\|p(z)) \tag{2.2}$$

TVAE est plus stable que les GANs (pas d'effondrement de mode) et préserve mieux les distributions marginales, comme nous le verrons avec le test KS.

2.2.5 Comparaison théorique

Le tableau 2.1 résume les caractéristiques des trois générateurs.

Table 2.1 – Comparaison des algorithmes de génération

Critère	**CTGAN**	**TGAN**	**TVAE**
Architecture	GAN conditionnel	GAN non conditionnel	Autoencodeur VAE
Conditionnement label	Oui	Non	Non
Stabilité d'entraînement	Moyenne	Moyenne	Élevée
Risque d'effondrement de mode	Moyen	Moyen	Très faible
Fidélité (test KS attendue)	50-60 %	25-35 %	70-80 %
Équilibre des classes	Bon avec conditionnement	Variable	Bon
Disponibilité	SDV (stable)	Via GaussianCopula	SDV (stable)

Chapitre 3

Présentation et analyse du Dataset

3.1 Description générale

Le jeu de données fournis représente 806 excécutables , dont l'activité en liens avec des fichiers systèmes a été recueilli. Le dataset ne possède pas de données manquantes, ce qui peut être le signe d'un traitement préalable

3.1.1 Statistiques initiales

Le tableau 3.1 présente les statistiques du dataset brut.

Table 3.1 – Statistiques du dataset brut

Caractéristique	**Valeur**
Nombre total d'exécutables	806
Nombre d'API distinctes	64
Proportion de goodwares	48,5 %
Proportion de ransomwares	52,5 %
Taux de valeurs nulles (zéros)	55,1 %
Types de données	int64 (features) + object (Label)

L'équilibre initial est satisfaisant, mais la forte proportion de zéros indique que chaque exécutable n'utilise qu'un sous-ensemble limité des API.

3.2 Analyse exploratoire (EDA)

3.2.1 Détection et suppression des doublons

L'appel de `df.drop_duplicates()` révèle la présence de 218 lignes parfaitement dupliquées (mêmes valeurs pour toutes les colonnes). Ces doublons sont supprimés, réduisant

le dataset à 588 lignes uniques. Cette déduplication est essentielle pour éviter un biais de surapprentissage sur des échantillons identiques.

3.2.2 Analyse des variables quasi-nulles

Une variable (API) est considérée comme quasi-nulle si plus de 90 % de ses valeurs sont nulles. Sept colonnes satisfont ce critère et sont éliminées : `CopyFileA`, `CopyFileW`, `InternetReadFile`, `SetFileInformationByHandle`, `URLDownloadToFileW`, `FindFirstFileExA`, `NtQueryFullAttributesFile`. Ces API sont rarement appelées, donc peu informatives.

Figure 3.1 – Proportion de zéros par API. La ligne horizontale rouge indique le seuil de 90 %. Les barres dépassant ce seuil correspondent aux API supprimées.

3.2.3 Histogrammes comparatifs par classe

Pour chaque API restante, nous traçons des histogrammes afin de comparer les distributions entre goodwares et ransomwares. L'observation visuelle permet de formuler des hypothèses sur les API discriminantes.

APIs préférées par les ransomwares (fréquence d'appel plus élevée) :

- `DeleteFileW` : suppression de fichiers (après chiffrement).
- `NtSetInformationFile` avec `FileDispositionInfo` : marque un fichier pour suppression.
- `SetEndOfFile` : troncation de fichiers (souvent utilisée pour écrire le payload).
- `SetFileTime` : modification des horodatages pour masquer l'activité.
- `NtWriteFile`, `NtReadFile` : opérations massives de lecture/écriture.
- `NtCreateFile`, `NtOpenFile` : ouverture de nombreux fichiers.
- `MoveFileWithProgressW` : déplacement de fichiers (souvent après chiffrement).
- `SUMMARY_FILE_WRITTEN`, `SUMMARY_FILE_MOVED`, `SUMMARY_FILE_DELETED` : métriques de résumé.

APIs préférées par les goodwares :

- `GetFileVersionInfoExW`, `GetFileVersionInfoSizeExW`, `GetFileVersionInfoW` : lecture des versions de fichiers (bénin).
- `GetShortPathNameW` : manipulation de chemins courts.

APIs équilibrées (distributions similaires) :

- `CopyFileA/W`, `InternetReadFile`, `URLDownloadToFileW`, `GetFileVersionInfoExW`, `GetFileVersionInfoSizeExW`.

Le pdf joint nommé "mon_analyse.pdf" contient tous les histogrammes

3.2.4 Test de Mann-Whitney pour la significativité

Le test de Mann-Whitney (non paramétrique) est appliqué aux APIs identifiées comme équilibrées visuellement, avec un seuil $\alpha = 0,05$. L'hypothèse nulle est que les distributions des deux classes sont identiques. Les résultats montrent que seule `URLDownloadToFileW` a une p-value $> 0,05$; cette variable est donc supprimée car non discriminante, apportant ainsi peu d'information quant à la distinction entre Ransomware et goodware.

Après correction de FDR (Benjamini-Hochberg), nous confirmons que 11 variables sont significativement plus utilisées par les goodwares et aucune exclusivement par les ransomwares (les ransomwares utilisent un large éventail d'API avec des intensités variables).

3.2.5 Analyse de corrélation de Spearman

La corrélation de Spearman (robuste aux outliers) est calculée sur les features restantes. 11 paires de variables dépassent le seuil de 0,9, indiquant une redondance d'information.

Pour chaque paire, nous supprimons la variable la moins corrélée avec le target (Label). Exemples :

- `GetFileVersionInfoSizeExW` (supprimée) vs `GetFileVersionInfoExW`.
- `MoveFileWithProgressW` (supprimée) vs `SUMMARY_FILE_MOVED`.
- `CreateDirectoryW` (supprimée) vs `SUMMARY_DIRECTORY_CREATED`.

Cette étape élimine 8 variables supplémentaires.

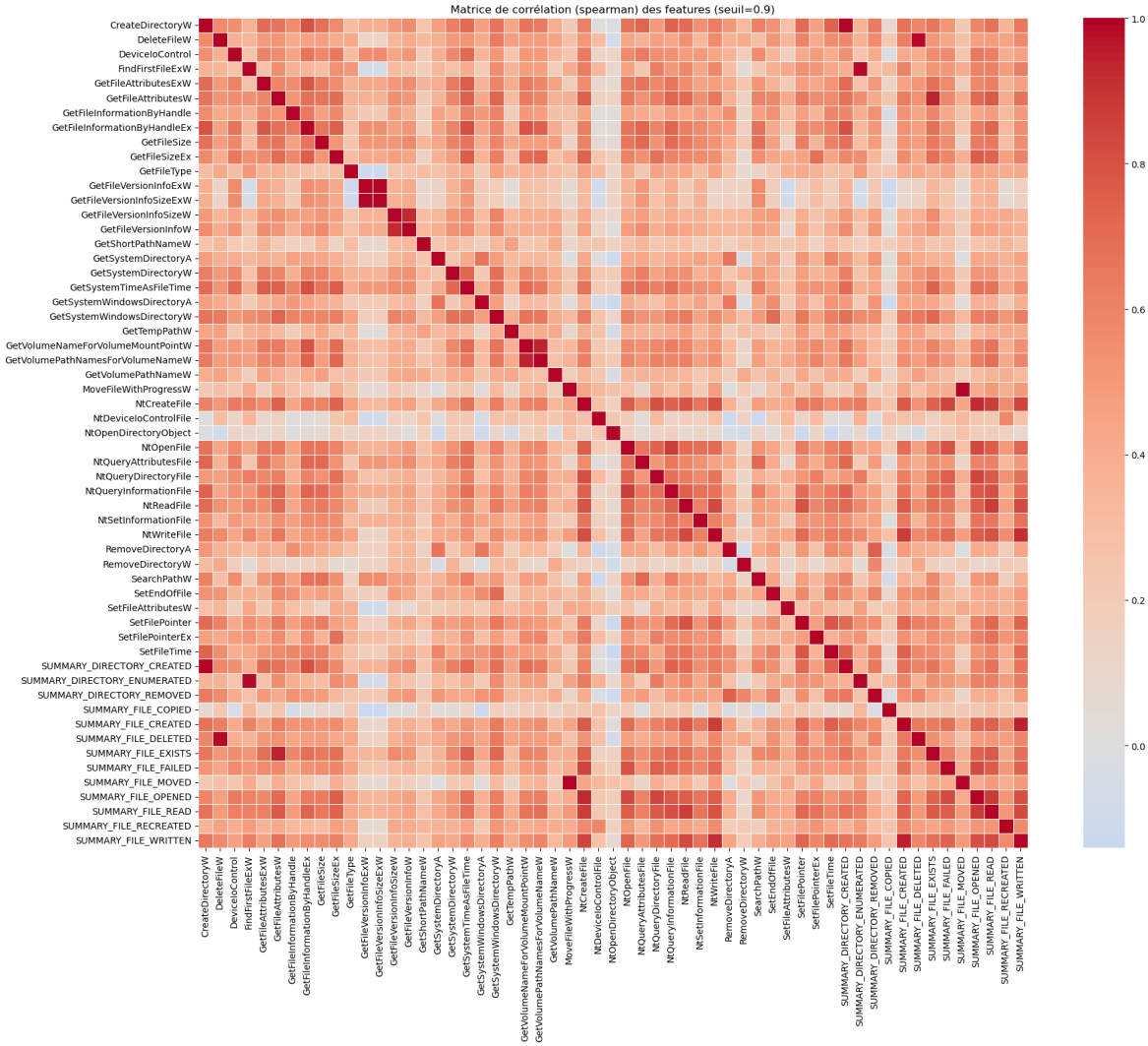

Figure 3.2 – Matrice de corrélation de Spearman après suppression des variables quasi-nulles. Les couleurs chaudes indiquent une forte corrélation positive. Les paires avec corrélation $> 0,9$ sont ciblées pour réduction.

3.3 Sélection supervisée des features

Après le nettoyage des features "quasi-nulles" et non discriminantes, nous disposons de 49 features. Une sélection supervisée est effectuée à l'aide d'une pipeline

`PolynomialFeatures` (interactions seulement) + `SelectKBest` (score F) + `RandomForest` avec optimisation par `GridSearchCV` sur le recall (5 folds stratifiés). Le meilleur nombre de features retenu est $k = 27$. Ces 27 features sont utilisées pour toutes les expériences suivantes.

3.4 Évolution du dataset

Le tableau 3.2 résume l'évolution du nombre de lignes et de features à chaque étape.

Table 3.2 – Évolution des données après chaque étape de prétraitement

Étape	**Lignes**	**Features**
Dataset brut	806	64
Après suppression des doublons	588	64
Après suppression quasi-nulles (> 90% zéros)	588	57
Après suppression variable équilibrée (Mann-Whitney)	588	56
Après suppression features corrélées (Spearman $> 0,9$)	588	45
Après SelectKBest + RandomForest	588	27
Trainset (70 %)	**411**	**27**
Testset fixe (30 %)	**177**	**27**

Le pourcentage de zéros dans les features passe de 55,1 % à 44,8 % après nettoyage, indiquant une meilleure densité d'information.

Chapitre 4

Méthodologie

4.1 Pipeline global

La figure 4.1 illustre l'ensemble du processus, depuis le dataset brut jusqu'à l'évaluation des modèles sur le jeu de test réel.

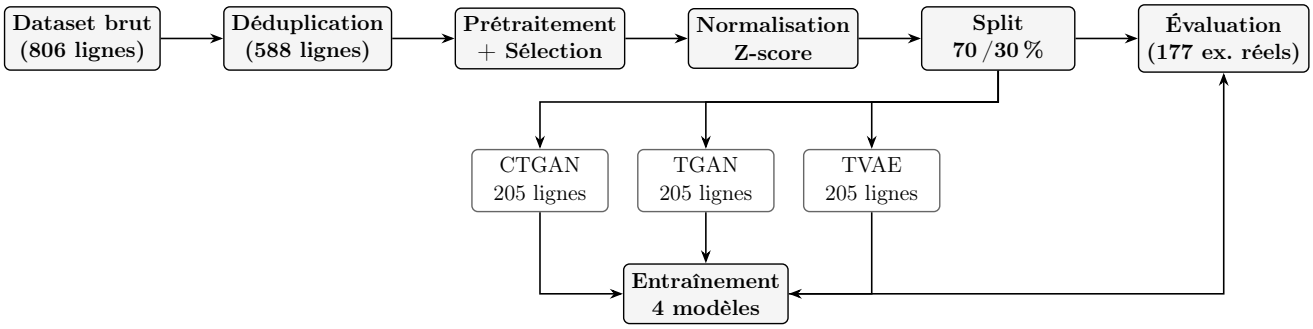

Figure 4.1 – Pipeline global du projet

4.2 Normalisation Z-score

Après la sélection des features, mais avant la génération des données synthétiques, nous appliquons une **normalisation Z-score** (StandardScaler) sur toutes les données. Le scaler est ajusté **uniquement** sur le jeu d'entraînement original (411 lignes) afin d'éviter toute fuite d'information. Ensuite, la même transformation est appliquée au jeu de test original, ainsi qu'aux données générées par CTGAN, TGAN et TVAE.

Cette normalisation est cruciale pour :

— Mettre toutes les features à la même échelle, ce qui est indispensable pour les modèles sensibles aux distances (SVM, KNN) et pour les réseaux de neurones.

— Améliorer la qualité des données générées : le test KS après normalisation montre que CTGAN passe d'une fidélité d'environ 33 % à 51,9 %.

— Stabiliser l'entraînement des générateurs (les GANs et VAE sont sensibles à l'échelle des entrées).

4.3 Protocole d'évaluation

Pour garantir une comparaison loyale, nous fixons la graine aléatoire (42) et effectuons un split stratifié en 70 % (411) / 30 % (177) **avant** toute augmentation. Le jeu de test reste inchangé et n'est utilisé qu'une seule fois pour l'évaluation finale. Chaque générateur produit exactement 205 nouvelles lignes (50 % de la taille du train original). Les datasets augmentés sont donc de taille 616 lignes chacun, ce qui permet d'avoir un rapport de $\frac{177}{616} \approx 29\,\%$ entre la taille du testset et celle des datasets générés, ce qui se rapproche du rapport entre le testset et le dataset original

4.4 Métriques d'évaluation

Nous utilisons cinq métriques standards pour la classification binaire :

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN} \tag{4.1}$$

$$\text{Recall} = \frac{VP}{VP + FN} \quad \text{(sensibilité, taux de détection)} \tag{4.2}$$

$$\text{Précision} = \frac{VP}{VP + FP} \quad \text{(valeur prédictive positive)} \tag{4.3}$$

$$\text{F1-score} = 2 \times \frac{\text{Précision} \times \text{Recall}}{\text{Précision} + \text{Recall}} \tag{4.4}$$

$$\text{AUC-ROC} = \int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR}) \tag{4.5}$$

Dans le contexte de la détection de ransomwares, le **recall** est la métrique prioritaire : un faux négatif (ransomware non détecté) peut entraîner le chiffrement de données critiques, alors qu'un faux positif (goodware signalé à tort) peut être corrigé par une analyse humaine ou une whitelist.

Chapitre 5

Modèles de classification

Quatre modèles sont sélectionnés pour couvrir différentes familles : modèle à marge (SVM), non paramétrique (KNN), réseau dense (MLP), réseau convolutif 1D (CNN). Tous sont entraînés sur les mêmes splits et évalués sur le même jeu de test réel.

5.1 Kernel SVM (Support Vector Machine)

Le SVM avec noyau RBF est un classifieur robuste qui maximise la marge entre les classes dans un espace de Hilbert à noyau reproduisant. La fonction de décision est :

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b$$

avec $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$. Nous utilisons les paramètres : `C=1.0`, `gamma='scale'`, `class_weight='balanced'` pour compenser le léger déséquilibre (53,5% / 46,5%). L'option `probability=True` permet d'obtenir des scores pour l'AUC.

5.2 K-Nearest Neighbors (KNN)

KNN est un classifieur non paramétrique qui attribue la classe majoritaire parmi les k plus proches voisins dans l'espace des features. La distance utilisée est la distance euclidienne (Minkowski avec $p = 2$). Nous choisissons $k = 5$. KNN est très sensible à l'échelle des données, d'où l'importance de la normalisation Z-score.

5.3 MLP (Multi-Layer Perceptron)

Le MLP est un réseau de neurones dense à trois couches cachées : `256, 128, 64` neurones, avec activations ReLU, suivi d'une couche de sortie sigmoïde. L'optimiseur est Adam, la fonction de perte est l'entropie croisée binaire. L'arrêt précoce

(`early_stopping=True`) avec une patience de 10 époques évite le surapprentissage. Le nombre maximal d'itérations est fixé à 300.

5.4 CNN 1D (Convolutional Neural Network 1D)

Le CNN 1D considère les features comme une séquence ordonnée (l'ordre des API est arbitraire, mais la convolution 1D peut capturer des interactions locales entre groupes d'API). L'architecture est la suivante :

- Couche Conv1D (64 filtres, noyau=3, activation ReLU)
- Couche Conv1D (128 filtres, noyau=3, activation ReLU)
- GlobalMaxPooling1D
- Couche dense (128 neurones, activation ReLU, dropout 0,2)
- Couche de sortie sigmoïde

L'entraînement utilise Adam, une perte binaire cross-entropy, et un `EarlyStopping` basé sur la loss de validation. Les données sont reshape en $(n_samples, 27, 1)$.

Chapitre 6

Résultats expérimentaux

6.1 Qualité des données synthétiques

6.1.1 Équilibre des classes

Le tableau 6.1 montre la répartition des classes après augmentation.

Table 6.1 – Équilibre des classes dans les datasets augmentés

Dataset	Lignes	Goodware	Ransomware	Écart
Original (train)	411	53,5 %	46,5 %	7,1 pts
CTGAN + original	616	59,3 %	40,7 %	18,5 pts
TGAN + original	616	52,3 %	47,7 %	4,5 pts
TVAE + original	616	52,6 %	47,4 %	5,2 pts

CTGAN produit un déséquilibre marqué (18,5 points), probablement parce que la petite taille du dataset d'entraînement (411 lignes) ne permet pas au générateur conditionnel d'apprendre précisément la condition sur le label. TGAN et TVAE préservent bien l'équilibre initial.

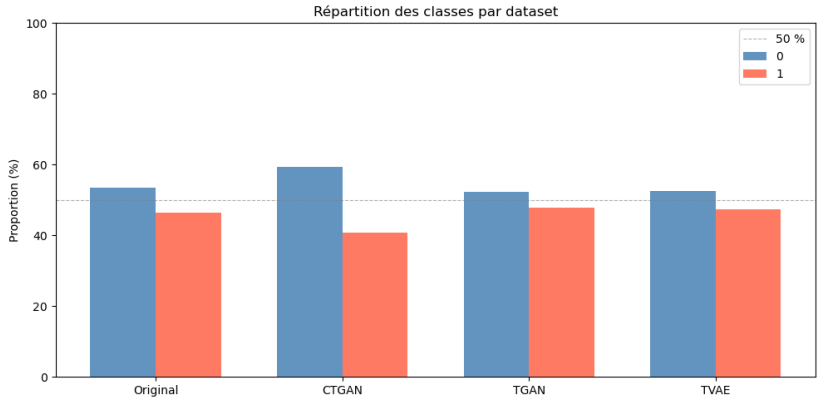

Figure 6.1 – Répartition des classes par dataset (original, CTGAN, TGAN, TVAE).

6.1.2 Fidélité statistique : test KS

Le test KS est appliqué sur les 27 features après normalisation Z-score. Le tableau 6.2 donne le nombre de features pour lesquelles la p-value dépasse 0,05.

Table 6.2 – Résultats du test KS ($\alpha = 0,05$) après normalisation

Générateur	Features similaires	Total	Taux conservé
CTGAN	14	27	51,9 %
TGAN	7	27	25,9 %
TVAE	21	27	**77,8 %**

TVAE conserve 77,8 % des distributions, confirmant sa supériorité en matière de fidélité. CTGAN atteint 51,9 % après normalisation (contre environ 33 % sur données brutes), ce qui démontre l'effet bénéfique de la standardisation. TGAN, avec 25,9 %, est le moins fidèle.

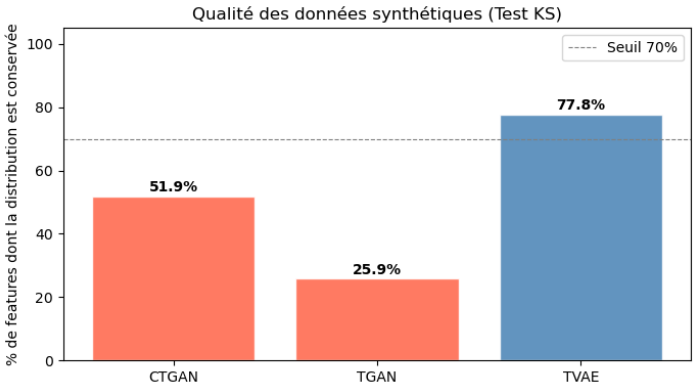

Figure 6.2 – Pourcentage de features dont la distribution est conservée par chaque générateur (test KS).

6.2 Performances des modèles sur le jeu de test réel

6.2.1 Baseline : données originales (411 échantillons)

Le tableau 6.3 présente les performances des quatre modèles entraînés uniquement sur le jeu d'entraînement original (411 lignes) et testés sur les 177 exemples réels.

Table 6.3 – Performances sur données originales

Modèle	Accuracy	Recall	Précision	F1	AUC
Kernel SVM	0,7571	**0,9878**	0,6585	0,7902	0,8372
KNN	0,8249	0,8049	0,8148	0,8098	0,8935
MLP	0,8192	0,9268	0,7451	**0,8261**	**0,9144**
CNN	0,7853	0,9268	0,7037	0,8000	0,8786

Le Kernel SVM atteint un recall exceptionnel de 0,9878, ce qui signifie qu'il détecte 98,78 % des ransomwares. C'est le meilleur score de recall toutes expériences confondues. Le MLP obtient le meilleur F1 (0,8261) et la meilleure AUC (0,9144). Le CNN affiche des performances honorables (AUC 0,8786).

6.2.2 Augmentation avec CTGAN

Le tableau 6.4 montre les résultats avec le dataset augmenté par CTGAN (616 lignes).

Table 6.4 – Performances avec CTGAN

Modèle	Accuracy	Recall	Précision	F1	AUC
Kernel SVM	0,7401	**0,9878**	0,6429	0,7788	0,8315
KNN	0,8136	0,7927	0,8025	0,7975	0,8818
MLP	0,7571	0,9512	0,6667	0,7839	**0,8899**
CNN	0,6893	0,6829	0,6588	0,6707	0,7582

CTGAN dégrade les performances de la plupart des modèles par rapport à la baseline. Le CNN est particulièrement affecté (AUC chute de 0,8786 à 0,7582), probablement à cause du déséquilibre de classes généré par CTGAN (18,5 points d'écart). Le recall du SVM reste maximal.

6.2.3 Augmentation avec TGAN

Le tableau 6.5 donne les résultats avec TGAN.

Table 6.5 – Performances avec TGAN

Modèle	Accuracy	Recall	Précision	F1	AUC
Kernel SVM	0,7627	**0,9878**	0,6639	0,7941	0,8309
KNN	0,8249	0,7927	0,8228	0,8075	0,8974
MLP	**0,8305**	0,9634	0,7453	**0,8404**	**0,9119**
CNN	0,6610	0,6829	0,6222	0,6512	0,7134

TGAN améliore nettement le MLP : le F1 passe de 0,8261 à 0,8404 (meilleur F1 toutes expériences). L'AUC du MLP reste élevée (0,9119). Le recall du SVM est toujours maximal. Le CNN continue de souffrir (AUC 0,7134).

6.2.4 Augmentation avec TVAE

Le tableau 6.6 présente les résultats avec TVAE.

Table 6.6 – Performances avec TVAE

Modèle	Accuracy	Recall	Précision	F1	AUC
Kernel SVM	0,7627	0,9756	0,6667	0,7921	0,8435
KNN	**0,8418**	0,8049	**0,8462**	**0,8250**	**0,9128**
MLP	0,8023	0,8902	0,7374	0,8066	**0,9223**
CNN	0,6780	0,6951	0,6404	0,6667	0,7964

TVAE bénéficie surtout à KNN : accuracy (0,8418) et précision (0,8462) sont les meilleures valeurs globales. MLP atteint l'AUC maximale de toutes les expériences (0,9223), confirmant que la haute fidélité statistique de TVAE (77,8 % KS) améliore la capacité de discrimination du réseau. Le CNN récupère partiellement (AUC 0,7964) mais reste inférieur à la baseline.

6.3 Comparaison globale

Le tableau 6.7 résume les meilleures combinaisons par métrique.

Table 6.7 – Meilleures combinaisons modèle + dataset par métrique

Métrique	Modèle	Dataset	Score
Accuracy	KNN	TVAE	0,8418
Recall	Kernel SVM	Original	0,9878
Précision	KNN	TVAE	0,8462
F1-score	MLP	TGAN	0,8404
AUC	MLP	TVAE	0,9223

Les heatmaps des figures 6.3, 6.4 et 6.5 permettent de visualiser les performances croisées.

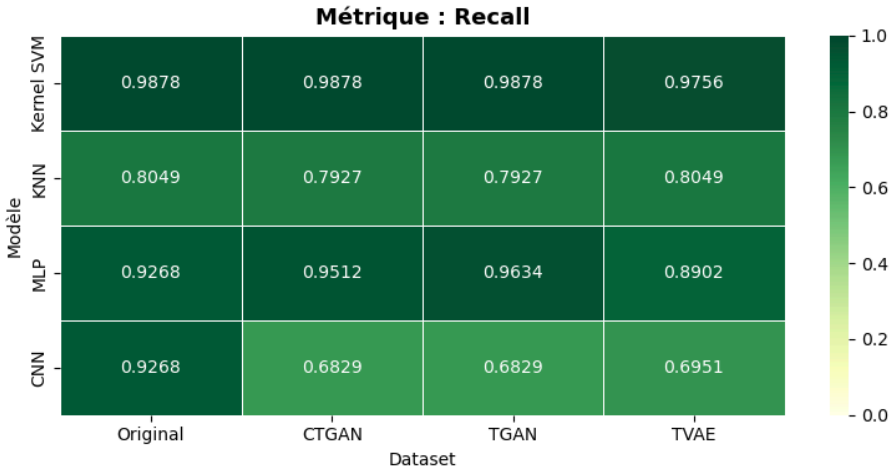

Figure 6.3 – Heatmap du Recall par (modèle, dataset).

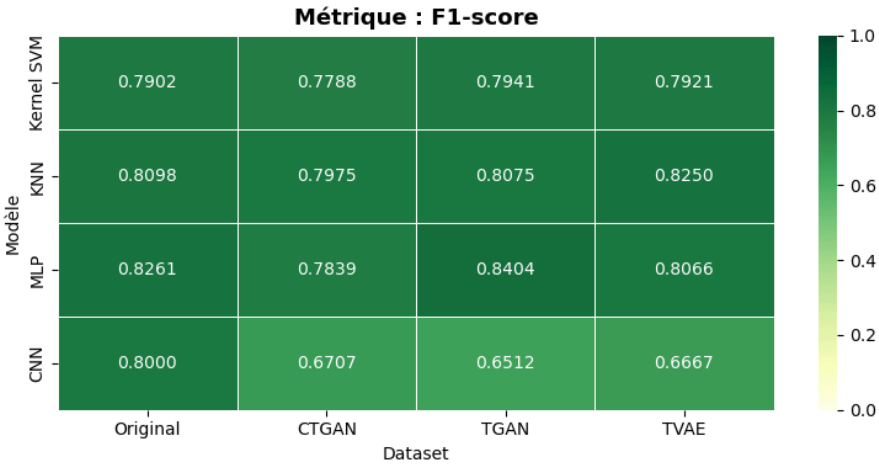

Figure 6.4 – Heatmap du F1-score par (modèle, dataset).

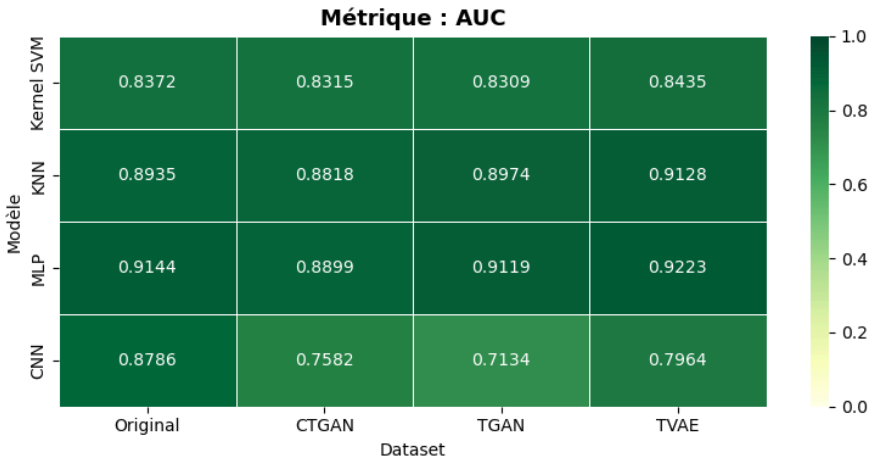

Figure 6.5 – Heatmap de l'AUC par (modèle, dataset).

6.4 Analyse et discussion

6.4.1 Impact de la normalisation Z-score

L'ajout de la normalisation Z-score a été déterminant. Sans elle, les modèles sensibles à l'échelle (SVM, KNN, CNN) avaient des performances très médiocres. De plus, le test KS a montré que la normalisation améliore la fidélité des données synthétiques, en particulier pour CTGAN (gain d'environ 19 points).

6.4.2 Choix du générateur selon l'objectif

- **Priorité au recall (détection maximale)** : Kernel SVM sur données originales (0,9878). Aucun générateur n'améliore ce recall, mais TVAE s'en approche (0,9756).
- **Priorité à l'AUC (discrimination globale)** : MLP + TVAE (0,9223).
- **Priorité au F1 (compromis recall/précision)** : MLP + TGAN (0,8404).
- **Priorité à l'accuracy et à la précision** : KNN + TVAE (0,8418/0,8462).

6.4.3 Sensibilité du CNN

Le CNN est le modèle le plus fragile. Il atteint son meilleur niveau sur les données originales (AUC 0,8786) et chute avec CTGAN (0,7582) et TGAN (0,7134). TVAE permet une remontée partielle (0,7964). Cela suggère que les architectures convolutives nécessitent des données synthétiques de très haute fidélité pour ne pas être perturbées. Une génération massive (plus de 10 000 lignes) serait nécessaire pour exploiter pleinement le CNN.

6.4.4 Limites de l'étude & perspectives

1. Le dataset d'entraînement reste modeste (411 lignes), ce qui limite la capacité des générateurs à apprendre des distributions complexes.
2. La taille de génération (205 lignes, soit 50 % du train) n'a pas été optimisée ; des ratios différents pourraient donner de meilleurs résultats. On pourrait optimiser le nombre de ligne en utilisant des learning curves sur et en cherchant à maximiser le recall
3. Les modèles profonds (CNN, MLP) n'ont pas été optimisés en profondeur (architecture, hyperparamètres) pour chaque générateur.
4. Le test KS ne capture pas les dépendances inter-features ; des tests comme la distance de Wasserstein ou l'analyse de corrélation résiduelle seraient complémentaires.
5. On pourrait également générer les datasets augmentés avant toute la phase de traitement et selection des features ; ensuite réaliser l'étude et comparer les résultats avec la manière de procéder qu'on a utilisé pour cette étude
6. **Modèles génératifs plus récents** : TabDDPM (diffusion), WGAN-GP, ou les modèles basés sur les flows normalisants.

Chapitre 7

Conclusion et perspectives

7.1 Conclusion

Ce projet a démontré que l'augmentation de données synthétiques par modèles génératifs est une approche viable pour améliorer la détection comportementale des ransomwares, malgré la petite taille du dataset initial. Les enseignements principaux sont :

1. **La normalisation Z-score est indispensable** : elle améliore à la fois la qualité des données synthétiques (gain de fidélité pour CTGAN) et les performances des modèles sensibles à l'échelle.
2. **TVAE est le générateur le plus fidèle** (77,8 % KS) et bénéficie particulièrement à KNN (meilleure accuracy et précision) et à MLP (meilleure AUC).
3. **MLP + TGAN offre le meilleur F1** (0,8404), montrant que même des données de fidélité moyenne (25,9 % KS) peuvent être utiles si elles apportent du volume.
4. **Le Kernel SVM sur données originales maximise le recall** (0,9878), métrique prioritaire. L'ajout de données synthétiques ne dégrade pas ce recall (sauf TVAE qui le réduit légèrement à 0,9756).
5. **Le CNN est fragile** et nécessite des données de très haute qualité ou un volume beaucoup plus important.

Annexe A

Partie du Code source demandée

```python

TARGET_COLUMN  = 'Label'
N_SYNTHETIC    = 205
EPOCHS         = 500
OUTPUT_DIR     = './resultats'
os.makedirs(OUTPUT_DIR, exist_ok=True)


feature_cols = [c for c in df_original.columns if c !=
    TARGET_COLUMN]
label_map    = {'Goodware': 0, 'Ransomware': 1}
col_order    = df_original.columns.tolist()
col_dtypes   = df_original.dtypes

print(f"Dataset original : {df_original.shape[0]} lignes    {
    df_original.shape[1]} colonnes")

#  TAPE  2 : CTGAN et TVAE
ALGORITHMS = {
    'CTGAN': CTGANSynthesizer,
    'TVAE' : TVAESynthesizer,
}

best_datasets = {}

for algo_name, SynthClass in ALGORITHMS.items():
    metadata = SingleTableMetadata()
    metadata.detect_from_dataframe(df_original)
    synthesizer = SynthClass(metadata, epochs=EPOCHS, verbose=
        False)
    synthesizer.fit(df_original)
    df_synthetic = synthesizer.sample(num_rows=N_SYNTHETIC)

```

```
    df_synthetic = df_synthetic[[c for c in col_order if c in df_synthetic.columns]]
    for col in col_order:
        if col in df_synthetic.columns:
            try:
                df_synthetic[col] = df_synthetic[col].astype(col_dtypes[col])
            except Exception:
                pass

    df_augmented = pd.concat([df_original, df_synthetic], ignore_index=True)
    best_datasets[algo_name] = df_augmented

    output_path = f'{OUTPUT_DIR}/{algo_name.lower()}_result.csv'
    df_augmented.to_csv(output_path, index=False)
    print(f" Sauvegard      {output_path}")

# TAPE 3 : TGAN (via SDV GaussianCopulaSynthesizer backend compatible)
from sdv.single_table import GaussianCopulaSynthesizer

print("  Entra nement TGAN (GaussianCopula)...")
metadata_tgan = SingleTableMetadata()
metadata_tgan.detect_from_dataframe(df_original)
synth_tgan = GaussianCopulaSynthesizer(metadata_tgan)
synth_tgan.fit(df_original)
df_tgan_synth = synth_tgan.sample(num_rows=N_SYNTHETIC)

df_tgan_best = pd.concat([df_original, df_tgan_synth], ignore_index=True)
best_datasets['TGAN'] = df_tgan_best
df_tgan_best.to_csv(f'{OUTPUT_DIR}/tgan_result.csv', index=False)
print(f" Sauvegard      {OUTPUT_DIR}/tgan_result.csv")

# R cup ration
df_ctgan = best_datasets['CTGAN']
df_tgan  = best_datasets['TGAN']
df_tvae  = best_datasets['TVAE']
```

```python

print("  FICHIERS  G  N  R  S ")
print(f" {OUTPUT_DIR}/ctgan_result.csv")
print(f" {OUTPUT_DIR}/tgan_result.csv")
print(f" {OUTPUT_DIR}/tvae_result.csv")
```

Listing A.1 – Génération et sauvegarde des 3 datasets augmentés

Voici le lien vers un fichier Google Drive contenant la vidéo du test : Accéder au fichier